Explorando o Paralelismo em Random Forests: Desempenho, Fidelidade Preditiva e Eficiência Energética com OpenMP

Gabriel L. Bonavina[1], **Estevan T. Santos**[1],
Marcelo F. Ohno[1], **Vinicius M. Goulart**[1]

[1]Universidade Federal de São Paulo (Unifesp)

gabriel.bonavina@unifesp.br estevan.santos@unifesp.br
marcelo.ohno@unifesp.br vmgoulart07@unifesp.br

Abstract. *Random Forest aggregates decision trees trained on bootstrap samples. Sorting candidate splits at each internal node tends to be the costliest step of training, while the trees themselves are independent. This paper measures the strong scaling of one OpenMP implementation on four benchmarks (150 to 581,012 samples), from 1 to 32 threads, on a hybrid 12-core, 16-thread Intel Core i5-12500H, with repeated runs in shuffled order. The median training speedup reaches 7.36× on Letter Recognition and 5.51× on Covertype; on both, the training gain stops growing at about 16 threads, the number of logical processors (on Letter, by a narrow margin). Prediction scales further, up to 11.3×. The CPU package energy falls by up to 4.0× and the energy-delay product improves by up to 30×. No superlinear speedup held in every run. Test accuracy is identical for every thread count.*

Resumo. *O algoritmo Random Forest combina árvores de decisão induzidas sobre amostras bootstrap. A ordenação dos cortes candidatos em cada nó interno tende a ser a etapa mais custosa do treinamento, enquanto as árvores crescem de forma independente. Este trabalho mede o escalonamento forte de uma implementação OpenMP em quatro bases (150 a 581 mil instâncias), de 1 a 32 threads, em um Intel Core i5-12500H híbrido de 12 núcleos e 16 threads, com execuções repetidas em ordem embaralhada. A mediana do speedup de treino chega a 7,36× no Letter Recognition e 5,51× no Covertype. Nas duas, o ganho do treino deixa de crescer por volta de 16 threads, o número de processadores lógicos (no Letter, por margem mínima). A predição escala mais, até 11,3×. A energia do pacote da CPU cai até 4,0×, e o produto energia-atraso melhora até 30×. Nenhum speedup acima do linear se repetiu em todas as execuções. A acurácia de teste é idêntica com qualquer número de threads.*

1. Introdução

A classificação supervisionada mapeia instâncias com atributos numéricos a classes discretas, com ampla aplicação em diagnóstico médico, visão computacional e sensoriamento remoto. Entre os métodos baseados em comitês (*ensembles*), o Random Forest [Breiman 2001, Breiman et al. 1984] destaca-se pela alta acurácia preditiva e robustez ao sobreajuste por meio da agregação de múltiplas árvores de decisão.

A indução de florestas sobre bases volumosas impõe elevado custo computacional. O algoritmo exibe paralelismo de dois grãos: as árvores crescem de forma independente, mas a construção de cada nó exige ordenações

sucessivas. Este artigo avalia o escalonamento forte de uma implementação OpenMP [OpenMP Architecture Review Board 2018] quanto a tempo de execução, acurácia e Produto Energia-Atraso (EDP) [Gonzalez and Horowitz 1996], com a energia do pacote da CPU lida pelo Intel RAPL [David et al. 2010], em bases de 150 a 581.012 instâncias e com 1 a 32 threads. Cada configuração é repetida em ordem embaralhada, e os resultados trazem média, mediana e desvio padrão.

2. Random Forest

2.1. Árvore de decisão

O conjunto de treino compreende amostras (x_i, y_i), onde cada vetor $x_i \in \mathbb{R}^d$ representa uma instância de d atributos contínuos e $y_i \in \{1, \ldots, K\}$ denota sua classe entre K categorias. Uma árvore de classificação e regressão (CART) [Breiman et al. 1984] particiona o espaço de atributos recursivamente por meio de testes univariados da forma $x_f \leq \theta$, com $f \in \{1, \ldots, d\}$ e $\theta \in \mathbb{R}$. Instâncias que satisfazem a desigualdade seguem para o filho esquerdo, e as demais para o direito.

Em cada nó interno com subconjunto de amostras S, o corte ideal maximiza a homogeneidade das partições geradas, mensurada pela impureza de Gini. Sendo p_k a fração empírica de instâncias da classe k em S, $I_G(S) = 1 - \sum_{k=1}^{K} p_k^2$. Um nó puro exibe $I_G = 0$, enquanto a divisão equilibrada entre duas classes resulta em $I_G = 0{,}5$. Ao avaliar um candidato que divide S em S_L e S_R, calcula-se a impureza média ponderada $I_G(S, f, \theta) = \frac{|S_L|}{|S|} I_G(S_L) + \frac{|S_R|}{|S|} I_G(S_R)$, elegendo-se o corte de menor valor. Na implementação, ordenam-se os valores do atributo f nas linhas de S e os limiares posicionam-se no ponto médio entre valores adjacentes distintos. A recursão cessa se o nó for puro, se $|S| < n_{\min}$, ou na profundidade máxima D, atribuindo-se à folha a classe majoritária. Árvores profundas têm alta variância.

2.2. Da árvore à floresta

O Random Forest atenua essa variância combinando T árvores de decisão descorrelacionadas [Breiman 2001] por meio de dois mecanismos estocásticos: (i) *bootstrap aggregating (bagging)* [Breiman 1996], onde cada árvore é induzida sobre N amostras sorteadas com reposição (das quais cerca de 63,2% são distintas); e (ii) *subespaço aleatório de atributos*, avaliando-se em cada nó apenas $m \leq d$ atributos sem reposição (tipicamente $m \approx \lfloor \sqrt{d} \rfloor$, passível de calibração), o que evita a predominância de atributos fortes.

Na inferência, a amostra percorre cada árvore até uma folha, definindo a predição por voto majoritário simples. Como as árvores não trocam dados durante o crescimento, o laço de indução é embaraçosamente paralelo. A reprodutibilidade decorre do gerador xorshift32 com semente privativa $s_t = s + t \cdot \texttt{0x9E3779B9} + 1$ para a árvore t, com $s = 42$ também na partição holdout estratificada (80% treino e 20% teste).

3. Paralelização com OpenMP

A implementação paraleliza dois laços e mantém sequencial o trabalho interno de cada um. O tamanho do time é `OMP_NUM_THREADS`. Com `OMP_DYNAMIC` desligado, o runtime não reduz esse time durante a execução. A cláusula `default(none)` obriga a classificar toda variável do laço como compartilhada ou privativa, de modo que as escritas fiquem explícitas.

3.1. Crescimento das árvores

O treinamento percorre as T árvores com `#pragma omp parallel for schedule(dynamic) default(none)` sobre o índice t. Sem tamanho de bloco, o escalonamento dinâmico entrega uma árvore por vez: a thread que termina pede o próximo índice. Cada árvore sorteia a própria amostra bootstrap e a própria sequência de atributos, então o número de nós e o tempo de cada ordenação variam. Um `schedule(static)` fixaria um bloco de índices no início do laço, e a thread que recebesse várias árvores profundas seguraria a barreira final enquanto as outras já tivessem terminado.

No caminho de sucesso, o que é compartilhado é somente leitura: o vetor de árvores, a matriz de treino e os hiperparâmetros. O índice t e a semente $s_t = s + t \cdot$ `0x9E3779B9` $+ 1$ são privativos. A rotina que cresce a árvore t aloca, na thread que a executa, o vetor bootstrap, os contadores de classe e a arena de nós daquela árvore. A ordenação de um atributo usa um vetor de pares local. Nenhuma dessas estruturas é compartilhada, e o crescimento não entra em região crítica. A única escrita compartilhada é a bandeira de falha de alocação, atualizada com `#pragma omp atomic write`. Na execução normal, a única sincronização é a barreira implícita no fim do `parallel for`.

A semente depende do índice da árvore, não da thread que a constrói. Mudar o escalonamento troca o núcleo que produz a árvore t, e preserva a sequência de cortes associada a esse índice.

O laço interno, que percorre os m atributos do nó e ordena os pares, permanece sequencial. Um segundo `parallel for` aninhado repartiria um vetor curto entre threads que o laço externo já distribuiu pelos núcleos. Por omissão, o runtime também não forma um segundo time dentro de uma região que já está ativa.

3.2. Predição e acurácia

Depois do treinamento os nós não mudam. A predição paraleliza as linhas, não as árvores, com `#pragma omp parallel for schedule(static) default(none)`. Cada linha desce todas as árvores sobre estruturas somente leitura, num caminho de comprimento no máximo D. Como esse custo é regular, o escalonamento estático divide as n linhas em blocos contíguos de cerca de n/p linhas, um por thread. O voto de cada linha percorre as árvores em série num vetor privativo. A acurácia usa outro laço estático com `reduction(+:ok)`.

3.3. Linha de base

Todos os speedups usam este mesmo binário. Com uma thread o `parallel for` continua no código, mas o time tem um único membro. Assim, a razão $T(1)/T(p)$ mede só o efeito do número de threads. Quando p passa de 16, as threads excedentes dividem os 16 processadores lógicos por fatias de tempo do sistema operacional.

4. Metodologia experimental

Os conjuntos de dados são Iris [Fisher 1936], Wisconsin Diagnostic Breast Cancer (WDBC) [Street et al. 1993], Letter Recognition [Frey and Slate 1991] e Forest Covertype [Blackard and Dean 1999]. Os hiperparâmetros de Iris, WDBC e Letter foram

calibrados por busca em grade, com a acurácia de teste média sobre várias sementes de partição. Em Iris e WDBC, entre as configurações a até 0,0025 da melhor acurácia média, escolheu-se a de mais árvores e, entre essas, a de menor tempo de treino. No Letter adotou-se a configuração de maior acurácia média da grade. O Covertype foi fixado em 50 árvores, profundidade 30 e $m = 18$. A Tabela 1 resume as dimensões, o holdout estratificado por classe (80% treino e 20% teste) e a acurácia de teste medida com uma thread.

Tabela 1. Partição holdout de 20% e acurácia de teste com uma thread. O número mínimo de amostras para partição ($n_{\min}$) é 5 para Iris e WDBC, e 2 para Letter e Covertype.

Conjunto	Treino	Teste	d	K	Árv.	Prof.	m	Acurácia
Iris	120	30	4	3	300	12	2	0,9333
WDBC	455	114	30	2	300	6	3	0,9912
Letter	16.000	4.000	16	26	300	30	4	0,9617
Cover	464.809	116.203	54	7	50	30	18	0,9599

A varredura de escalonamento forte usa o mesmo binário OpenMP (GCC 13.2, MinGW-w64, `-O2`) com `OMP_NUM_THREADS` em $\{1, 2, 4, 8, 12, 16, 32\}$, `OMP_DYNAMIC` desligado e semente 42. O hardware é um notebook com Intel Core i5-12500H e 16 GB de DDR5-4800, sob Windows 11, ligado à tomada e no plano de energia Equilibrado. O processador é híbrido: 4 núcleos de desempenho (P), com *hyper-threading* e turbo de até 4,5 GHz, e 8 núcleos de eficiência (E), sem *hyper-threading* e com turbo de até 3,3 GHz. São 12 núcleos, 16 threads lógicas, 18 MB de cache L3 e potência base de 45 W. Os pontos 4, 8, 12 e 16 da varredura coincidem, nominalmente, com o número de núcleos P, de threads lógicas dos núcleos P, de núcleos físicos e de threads lógicas. O ponto de 32 excede as threads lógicas.

Cada base roda em n blocos: $n = 30$ em Iris e WDBC, $n = 10$ no Letter e $n = 5$ no Covertype. As bases rodaram nessa ordem, num total de 525 execuções registradas uma a uma. Um bloco executa uma vez cada número de threads, numa ordem embaralhada com semente registrada (20261103). Assim, a execução com uma thread, que é o denominador de todos os speedups, não ocupa sempre a mesma posição na sessão. Cada execução é um processo novo, com pausa de 1 s entre execuções, e uma execução de aquecimento é descartada antes do primeiro bloco de cada base. Não houve controle de afinidade. O libgomp desta toolchain ignora `OMP_PLACES` e `OMP_PROC_BIND` (*affinity not supported*), e o escalonador do Windows decide em que núcleo, P ou E, cada thread roda.

Para cada configuração reportamos média, mediana e desvio padrão amostral. O speedup de uma etapa é a razão entre a mediana do tempo com uma thread e a mediana com p threads. Dividir a mediana de uma thread pelo tempo de cada execução com p threads dá ainda a faixa de speedups individuais, que mostra se um ganho se repete. Fixamos dois critérios antes de olhar os dados. Um speedup acima do linear só é afirmado se aparecer em todas as execuções. O ganho é considerado esgotado entre dois pontos quando a mediana do tempo muda menos de 5% e o coeficiente de variação (desvio padrão sobre média) fica abaixo de 10% nos dois pontos. Pontos com variação maior são tratados como indícios preliminares.

A energia E não é a do notebook inteiro. O Intel RAPL [David et al. 2010] acumula joules em domínios do chip. O domínio lido aqui é o *package*: núcleos, caches, controlador de memória e GPU integrada, no mesmo chip. O Windows expõe o contador pelo PDH em picowatt-hora, unidade que conferimos contra o contador de potência do mesmo conjunto. Ele é lido no início e no fim do processo; E é a diferença, convertida para joules. A DRAM permanece num domínio próprio do RAPL e não entra nessa diferença.

A potência média do pacote é $\bar{P} = E/\Delta t$, onde Δt vai do lançamento ao término do processo (leitura do arquivo, partição, treino e inferência). O Produto Energia-Atraso é $\mathrm{EDP} = E \cdot \Delta t$, e o speedup correspondente é $S_{\mathrm{EDP}} = E_1 \Delta t_1 / E_p \Delta t_p$. Como o EDP multiplica a redução de energia pela redução de tempo, S_{EDP} acima de p não indica ganho superlinear. Os tempos de treino e de predição vêm de cronômetros internos do programa (`QueryPerformanceCounter`, resolução abaixo de 1 μs). O tempo de predição cobre a inferência sobre os conjuntos de treino e de teste, usada no cálculo das duas acurácias. Em Iris e WDBC o processo inteiro dura menos de 0,2 s, e o coeficiente de variação da energia fica entre 13% e 75%. Por isso as conclusões energéticas apoiam-se em Letter e Covertype.

5. Resultados

A Figura 1 mostra a curva de speedup do treinamento, e a Tabela 2 lista mediana, média e desvio padrão dos tempos. Em Letter e Covertype o coeficiente de variação do treino não passa de 5%. Nenhum ponto de treino fica acima do linear: já com 2 threads o speedup é de 1,82× a 1,88×. Com 32 threads o treino acelera 7,36× no Letter (8,01 s para 1,09 s), 5,51× no Covertype (192,4 s para 34,9 s), 7,72× no WDBC e 3,43× no Iris.

Tabela 2. Tempo de treinamento (s; ms em Iris e WDBC): mediana $\tilde{t}$, média $\bar{t}$ e desvio padrão s de n execuções ($n = 30$, 30, 10 e 5), e speedup S pela razão das medianas.

Conjunto		1	2	4	8	12	16	32
Iris (ms)	$\tilde{t}$	10,4	5,7	3,6	3,0	2,6	2,6	3,0
	$\bar{t}$	10,4	5,8	3,7	3,1	2,7	2,8	3,0
	s	0,5	0,4	0,4	0,4	0,2	0,5	0,3
	S	1,00	1,82	2,88	3,44	3,93	3,99	3,43
WDBC (ms)	$\tilde{t}$	128,2	68,1	36,9	25,4	21,0	17,6	16,6
	$\bar{t}$	128,3	68,6	37,1	25,4	20,8	17,9	16,9
	s	2,1	3,1	1,1	1,1	1,5	1,1	1,2
	S	1,00	1,88	3,47	5,04	6,11	7,30	7,72
Letter	$\tilde{t}$	8,01	4,40	2,56	1,60	1,25	1,15	1,09
	$\bar{t}$	8,01	4,41	2,54	1,60	1,26	1,14	1,10
	s	0,09	0,07	0,06	0,02	0,06	0,03	0,05
	S	1,00	1,82	3,13	5,02	6,42	6,99	7,36
Cover	$\tilde{t}$	192,4	103,3	60,1	44,4	37,8	35,8	34,9
	$\bar{t}$	193,0	104,1	60,2	44,7	37,8	35,7	34,8
	s	1,8	1,1	0,6	0,6	0,2	0,5	0,5
	S	1,00	1,86	3,20	4,34	5,08	5,37	5,51

Pelo critério da Seção 4, o ganho do treino se esgota em 16 threads no Covertype:

de 16 para 32 a mediana cai 2,5%, com coeficiente de variação de 1,5%. No Letter, a queda de 16 para 32 é de 4,99%, com coeficiente de variação abaixo de 5%, e as faixas de speedup individual se sobrepõem (6,81× a 7,51× contra 6,59× a 7,60×). Pelo critério, o ganho do Letter também se esgota em 16 threads, mas por margem mínima. No WDBC o tempo ainda cai 5,5% de 16 para 32. No Iris, o máximo fica em 12 e 16 threads (3,93× e 3,99×), e 32 threads recuam para 3,43×. Esse recuo é preliminar: o coeficiente de variação chega a 17% com 16 threads, e as faixas individuais de 16 e 32 se sobrepõem.

O intervalo do processo inteiro, o mesmo Δt usado em $\bar{P}$ e no EDP, acelera menos que o treino porque inclui a leitura do arquivo e a partição holdout, que são sequenciais (cerca de 7 s no Covertype, com qualquer número de threads). No Covertype ele passa de 210,5 s para 42,8 s (4,92× em 32 threads); no Letter, de 9,86 s para 1,32 s (7,46×).

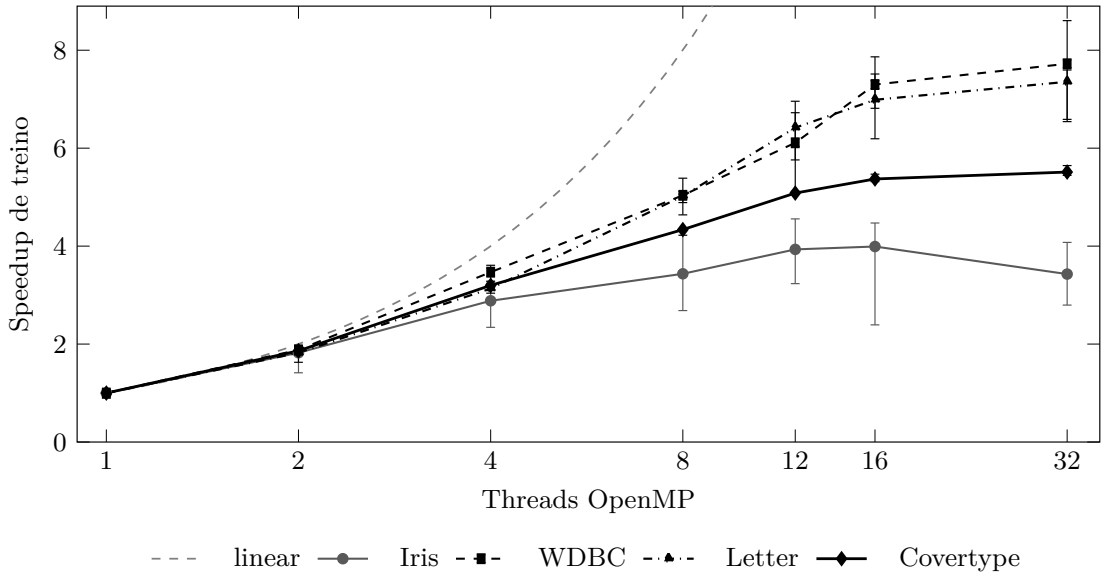


Figura 1. Speedup de treinamento (mediana) em relação à mesma implementação OpenMP com uma thread. As barras vão do menor ao maior speedup individual. A reta tracejada é o speedup linear; o eixo horizontal está em escala logarítmica.

A predição escala mais que o treino (Tabela 3): com 16 threads, Letter passa de 1,75 s para 0,16 s (10,88×) e Covertype de 11,38 s para 1,01 s (11,25×). No Covertype, de 16 para 32 a mediana muda 0,8%, com coeficiente de variação abaixo de 3%, e o ganho se esgota em 16 threads. No Letter a mediana muda 4,8% de 12 para 16 e 2% de 16 para 32, mas o coeficiente de variação chega a 12%. Logo, esse esgotamento é preliminar. Três medianas ficam pouco acima do linear: Covertype com 2 e 4 threads (2,05× e 4,14×) e Letter com 4 (4,15×). Pelo critério adotado, esse excedente não se confirma. Os menores speedups individuais desses pontos são 1,94×, 3,95× e 3,73×, abaixo do linear, e a diferença de 2 a 4% fica dentro da variação entre execuções. No Iris e no WDBC a predição dura de 0,4 a 4 ms e não entra na tabela.

A Tabela 4 reúne a energia do pacote e o speedup de EDP, e a Figura 2 compara os dois fatores. Com 32 threads, a energia cai de 243 J para 61 J no Letter e de 5.194 J para 1.837 J no Covertype. O EDP melhora 30,3× e 14,0×. A potência média do pacote sobe

Tabela 3. Tempo de predição (s), sobre os conjuntos de treino e de teste: mediana, média, desvio padrão e speedup pela razão das medianas.

Conjunto		1	2	4	8	12	16	32
Letter	$\tilde{t}$	1,75	0,88	0,42	0,24	0,17	0,16	0,16
	$\bar{t}$	1,77	0,88	0,42	0,25	0,17	0,16	0,17
	s	0,10	0,04	0,03	0,01	0,01	0,02	0,02
	S	1,00	1,98	4,15	7,14	10,36	10,88	10,67
Cover	$\tilde{t}$	11,38	5,55	2,75	1,59	1,15	1,01	1,00
	$\bar{t}$	11,41	5,60	2,77	1,60	1,16	1,01	0,99
	s	0,09	0,17	0,07	0,03	0,01	0,02	0,03
	S	1,00	2,05	4,14	7,15	9,89	11,25	11,34

de 24,6 W com uma thread para 46,5 W no Letter e 42,9 W no Covertype, e estabiliza a partir de 12 threads perto da potência base de 45 W. A energia deixa de cair a partir de 16 threads: de 16 para 32 ela muda 1,4% no Letter e 0,6% no Covertype. A acurácia de teste é idêntica em todas as execuções e em todos os números de threads: 0,9333 no Iris, 0,9912 no WDBC, 0,9617 no Letter e 0,9599 no Covertype.

Tabela 4. Energia do pacote da CPU (J): mediana, média e desvio padrão, e speedup de EDP pela razão das medianas. Iris e WDBC não são listados (ver Seção 4).

Conjunto		1	2	4	8	12	16	32
Letter	$\tilde{E}$	243,3	156,9	105,6	81,1	68,7	62,2	61,3
	$\bar{E}$	246,6	157,3	105,5	81,3	68,7	62,1	61,8
	s	9,3	7,8	5,6	2,7	4,0	3,6	3,5
	S_{EDP}	1,00	2,91	7,75	15,56	24,22	28,68	30,30
Cover	$\tilde{E}$	5.194	3.336	2.410	2.174	1.944	1.849	1.837
	$\bar{E}$	5.270	3.411	2.401	2.174	1.950	1.867	1.837
	s	189	117	35	27	20	37	28
	S_{EDP}	1,00	2,85	6,54	9,52	12,30	13,60	14,02

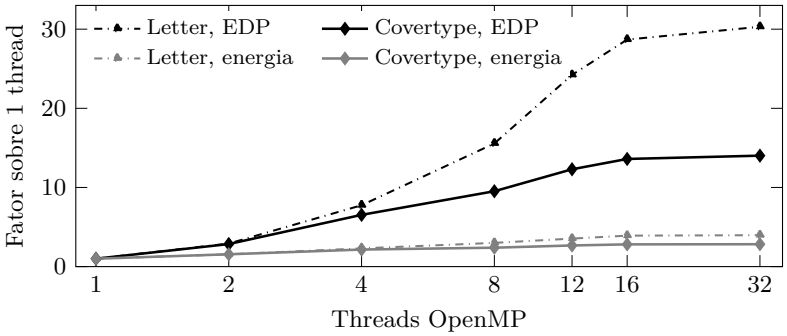


Figura 2. Redução da energia do pacote (E_1/E_p, em cinza) e speedup de EDP (em preto) sobre a execução com uma thread, em Letter e Covertype (medianas). O EDP cresce mais que a energia porque multiplica a redução de energia pela redução de tempo. As duas curvas ficam quase planas acima de 16 threads.

6. Discussão

O treino fica abaixo do linear desde 2 threads. A potência medida sugere uma explicação, que não verificamos. Com uma thread o pacote gasta 24,6 W, e a partir de 8 a 12 threads chega perto dos 45 W de potência base; é provável que, nesse ponto, os núcleos passem a rodar a uma frequência menor. Sem afinidade, parte das threads também pode cair em núcleos E, mais lentos, ou dividir um núcleo P pelo *hyper-threading*. A disputa das árvores pela cache L3 e pela memória é outra causa possível, que só contadores de hardware separariam. Acima de 16 threads não há processadores lógicos novos, e o ganho do treino se esgota nesse ponto em Letter e Covertype.

A predição escala mais que o treino nos mesmos núcleos: 9,89× contra 5,08× no Covertype com 12 threads. A predição reparte linhas de custo uniforme sobre árvores somente de leitura, enquanto o treino ordena atributos e depende do tamanho de cada árvore. Uma hipótese é que a descida nas árvores seja limitada pela latência de memória, e não pela frequência, e por isso sofra menos com núcleos mais lentos. Não medimos contadores de hardware para confirmá-la.

Nas bases pequenas o mesmo teto aparece mais cedo. O Iris, com 120 amostras e 10 ms de treino sequencial, para em cerca de 4×. Com 2 a 3 ms de treino, é provável que o custo de criar o time e repartir 300 árvores curtas pese tanto quanto o trabalho. O coeficiente de variação do treino é o maior entre as bases. O WDBC, com 455 amostras, ainda ganha até 32 threads, mas cada passo acrescenta pouco. Na energia, o tempo cai mais depressa que a potência sobe até 16 threads, e a energia total diminui. Acima disso a potência média estabiliza perto de 45 W e o tempo quase não muda, então mais threads não reduzem a energia.

As conclusões valem para um processador e um sistema operacional. Sem controle de afinidade, não sabemos quantas threads rodaram em núcleos P ou E em cada execução. Esse posicionamento é uma fonte de variação que a repetição mede, mas não remove. O Covertype tem só cinco execuções por ponto, e o Iris tem coeficiente de variação de até 17%.

7. Conclusão

A varredura de 1 a 32 threads, com execuções repetidas em ordem embaralhada, mostra que o Random Forest em OpenMP ganha com mais threads até o número de processadores lógicos. No i5-12500H, a mediana do speedup de treino chega a 7,36× no Letter e 5,51× no Covertype. A predição chega a 10,9× e 11,3×, e o EDP do pacote melhora até 30×, com a acurácia idêntica em todos os pontos. Em Letter e Covertype, o ganho de tempo de treino e de energia se esgota por volta de 16 threads, o número de threads lógicas (no Letter, por margem mínima). Nenhum speedup acima do linear se repetiu em todas as execuções.

O arranjo prático, neste processador, é usar 16 threads nas bases em que o treino dura segundos ou mais. Quando o treino dura poucos milissegundos, o speedup para em cerca de 4× e varia muito entre execuções. Trabalhos futuros podem fixar threads em núcleos P e E com a API do Windows, para separar o efeito do processador híbrido; usar um número mínimo de instâncias para disparar a paralelização nas bases pequenas; e paralelizar o laço interno de ordenação dos atributos, que deve compensar só em bases grandes, como o Covertype ($m = 18$).

Referências

Blackard, J. A. and Dean, D. J. (1999). Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables. *Computers and Electronics in Agriculture*, 24(3):131–151.

Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2):123–140.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.

Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth.

David, H., Gorbatov, E., Hanebutte, U. R., Khanna, R., and Le, C. (2010). RAPL: Memory power estimation and capping. In *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design*, pages 189–194.

Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188.

Frey, P. W. and Slate, D. J. (1991). Letter recognition using Holland-style adaptive classifiers. *Machine Learning*, 6(2):161–182.

Gonzalez, R. and Horowitz, M. (1996). Energy dissipation in general purpose microprocessors. *IEEE Journal of Solid-State Circuits*, 31(9):1277–1284.

OpenMP Architecture Review Board (2018). *OpenMP Application Programming Interface*. Version 5.0.

Street, W. N., Wolberg, W. H., and Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. In *Biomedical Image Processing and Biomedical Visualization*, volume 1905, pages 861–870. SPIE.